

Sustainable Farm

Visitor Management System - Technical Documentation

Backend, frontend, end-to-end connectivity, integration with the other modules of the platform, and the future authentication & role model.

Area	Summary
System	Visitor Management (agri-tourism: visits, registrations, bookings, events, feedback, staff)
Backend	Java 21, Spring Boot 4.1, REST API under /api/v1, Spring Data JPA / Hibernate, PostgreSQL
Frontend	React 19, Vite 8, Tailwind CSS v4, feature-folder module under /visitormanagement
Connectivity	Vite dev proxy /api to localhost:8080, CORS-whitelisted origins, shared apiClient
Current security	None enforced yet - open API, no login (see section 6 for the planned role model)
Testing	300 backend tests (H2 profile), frontend lint + production build green

This document reflects the work completed on the visitor management feature branch (frontend + dashboard + resilient backend), the way the module talks to the rest of the platform, and the role-based access rules that are planned once authentication is introduced.

1. Overview

The Visitor Management system organises the farm's agri-tourism activity. It covers the whole visitor journey: registering a visitor and their purpose, booking a slot, confirming attendance, briefing on safety, running educational workshops and events, collecting on-site and post-visit feedback, and tracking staff who participate. A dashboard aggregates weekly key figures and upcoming tasks for the farm's management.

The module is delivered as two independent applications that talk over HTTP/JSON: a Spring Boot REST API (the backend) and a React single-page application (the frontend). Neither one embeds the other; the browser loads the frontend, which calls the backend through a shared HTTP client.

2. Backend (Java / Spring Boot)

Built with **Java 21** and **Spring Boot 4.1**. The module lives in `com.infineonbit.sustainablefarm.modules.visitormanagement` and exposes a monolithic REST API under the `/api/v1` base path. Persistence is handled by Spring Data JPA (Hibernate) against a **PostgreSQL** database; schema changes are applied with `ddl-auto=update`.

2.1 Entity model

Entity	Purpose
Visitor	A person/group coming to the farm, with type (individual, group, school) and purpose (tourism, purchase, partnership, investment, education)
Registration	A visit request tied to a visitor and a time slot; lifecycle pending, confirmed, checked-in, rejected
TimeSlot	A date/time window offered for visits, with capacity and optional assigned guide
AgriActivity	Paid activities (e.g. standard tour, mango tasting, solar workshop)
Booking	An activity reservation linked to a registration/time slot; pending, confirmed, completed, with payment status
TourStop	Stops of the guided circuit (safety briefing, mango orchard, drip irrigation, solar plant, processing unit, wrap-up)
Workshop	Educational sessions (school discovery day, solar workshop, etc.)
Event	Open days, buyer visits and school days with capacity and statuses
Briefing	Tracks the delivery of the welcome & safety briefing per registration
Feedback	Satisfaction record collected on-site or later (email/SMS survey)
SurveySend	A survey that was sent to a visitor for post-visit feedback
Staff	Farm staff member; the StaffRole enum defines GUIDE, RECEPTION and MANAGER

2.2 API surface (controllers)

Controller	Base path	Main operations
VisitorController	<code>/api/v1/visitors</code>	CRUD + delete with reference safety (204/404/422)
RegistrationController	<code>/api/v1/registrations</code>	create, approve, reject, check-in, cancel

Controller	Base path	Main operations
TimeSlotController	/api/v1/time-slots	CRUD, availability, assign-guide, capacity > 10 supported
BookingController	/api/v1	/activities and /bookings CRUD, occupancy, pay, confirm, complete, cancel
BriefingController	/api/v1	read briefing, deliver briefing (safety signature)
EducationalProgramController	/api/v1	/tour-stops and /workshops CRUD, publish, deactivate
EventController	/api/v1/events	CRUD, publish, register attendees
StaffController	/api/v1/staff	CRUD
FeedbackController	/api/v1	/feedback list + on-site submit (by visitorName or visitorId), /feedback/summary, /surveys
DashboardController	/api/v1/dashboard	weekly KPIs, upcoming tasks and events

2.3 Services, jobs and cross-cutting

- **Scheduling:** SchedulingService enforces per-date serialisation so concurrent slot creation cannot overlap; a shared cap was lifted to allow capacities above 10.
- **Booking notifications:** BookingNotificationListener reacts to BookingNotificationEvent and sends confirmation + 24h-reminder emails (SMTP in production, console in dev); BookingReminderJob runs the lifecycle.
- **Visit lifecycle:** VisitLifecycleJob rolls registrations/check-ins forward based on the schedule.
- **Dashboard:** DashboardService aggregates average satisfaction, visitors this week, slots booked, pending briefings and upcoming events.
- **Error mapping:** a dedicated exception handler returns standard HTTP codes (400, 404, 422) with a consistent JSON error envelope.
- **CORS:** WebConfig whitelists the frontend origins `http://localhost:5173` and `http://localhost:3000`.
- **Dev reference data:** DevDataSeeder (dev profile only) seeds tour stops, workshops, activities, a fortnight of time slots, an event, visitors and registrations so screens are never empty when testing.

The module counts dedicated tests (VisitorControllerTest, FeedbackServiceImplTest, BookingServiceImplTest, Scheduling concurrency integration tests, repository tests, and more) run against an in-memory H2 database via the `application-test.properties` profile - 300 tests in total, all green.

3. Frontend (React / Vite)

Built with **React 19**, bundled by **Vite 8**, styled with **Tailwind CSS v4**. The module is a feature folder `frontend/src/features/visitormanagement` registered in the application router under the route `/visitormanagement`.

3.1 Screens

Screen	Route	What it does
DashboardPage	/visitormanagement	Weekly KPIs with animated counters, upcoming events and tasks

Screen	Route	What it does
SchedulingPage	/visitormanagement/scheduling	Weekly slot grid, create/edit/delete slots, capacity > 10, cancel
RegistrationPage	/visitormanagement/registration	Visitor form + registration queue against stored directory records
VisitorsPage	/visitormanagement/visitors	Directory of visitors; delete only when no registrations remain
EducationPage	/visitormanagement/education	Tour stops timeline and workshops management
SafetyPage	/visitormanagement/safety	Safety briefing tracker with a plain checkbox signature
BookingPage	/visitormanagement/booking	Activities and bookings, pay/confirm/complete/cancel, occupancy
FeedbackPage	/visitormanagement/feedback	On-site feedback by typed name + survey by email link
EventsPage	/visitormanagement/events	Event cards, publish, registrations
StaffPage	/visitormanagement/staff	Staff directory (roles include GUIDE, RECEPTION, MANAGER)

3.2 UX details delivered

- **Shared motion language** in index.css: page-in, fade-up, fade-in and row-in entrance animations driven only by opacity/transform, with staggered delays for table rows; prefers-reduced-motion disables the effects; custom keyframes were named vm-row-in to avoid clashing with the shared row-in animation.
- **useCountUp** hook animates the dashboard KPIs on mount without re-animating on refetch.
- **Circular logo**: the farm logo is clipped inside a white circle in the sidebar (empty corners masked by overflow-hidden).
- **Module shell**: VisitorManagementSidebar + ModuleTopBar provide the approved navigation (logo circle, module name, screen entries, logout).
- **Design tokens**: the frontend uses the shared Tailwind v4 @theme palette from develop (primary/navy/accent/success/warning/error), so the module automatically matches the rest of the platform.

4. Connectivity between frontend and backend

The two tiers communicate exclusively over HTTP/JSON.

Aspect	Mechanism
Dev proxy	Vite proxies every /api request to http://localhost:8080 (changeOrigin=true). The browser only ever talks to the Vite server.
CORS	Backend allows origins localhost:5173 (Vite) and localhost:3000 (Docker build) via app.cors.allowed-origins.
HTTP client	frontend/src/shared/api/client.js wraps axios with a 10s timeout and exposes get, post, put, patch, delete.
Module API	frontend/src/features/visitormanagement/api/visitormanagementApi.js maps every VM endpoint to apiClient verbs (22 GET, 19 POST, 8 PUT, 6 PATCH, 6 DELETE).

Aspect	Mechanism
Endpoints	Module endpoints are declared in /features/visitormanagement/api/endpoints.js (VM_ENDPOINTS). The shared endpoints.js stays empty on purpose to avoid merge conflicts.
Error handling	The shared client normalises errors; the backend returns a consistent JSON error body with the HTTP status.

End-to-end flow example: the BookingPage asks visitormanagementApi to POST /api/v1/bookings; axios sends the JSON to Vite (:5173), which forwards it to Spring Boot (:8080); BookingService validates, persists via JPA into PostgreSQL and returns the BookingResponse; if the request references a deleted or linked record, the exception handler answers 404/422 and the page surfaces the error inline.

Verification: 300 backend tests green (H2), npm run lint and npm run build green, live smoke-tested end-to-end (feedback by name, visitor deletion 204/404/422, slot capacity > 10, dashboard data served through the Vite proxy).

5. Integration with the other modules

Visitor Management is one module of a wider platform. The repository contains sibling modules - machinery, plants, sitesecurity, watersupply, energysupply, cropstorage, producttransformation, salesmarketing - each with its own feature folder on the frontend and its own Java subpackage on the backend. The VM module participates in the platform as follows:

- **Shared frontend conventions:** every module is a frontend/src/features/<module> folder mounted in the same router; the app-wide shared endpoints.js states that module endpoints must live in the module folder, exactly what visitormanagement/api/endpoints.js does.
- **Shared design tokens:** the VM screens reuse the global Tailwind @theme palette and fonts (Inter/Montserrat) defined in index.css, so the module visually merges with machinery, plants, sitesecurity, etc.
- **Shared components & utilities:** Modal, Pagination and EmptyState live under frontend/src/shared and are available to every module.
- **Shared HTTP client:** apiClient is the single axios wrapper for the whole SPA; the VM added the put verb once, benefiting every module.
- **Backend boundaries:** each module owns its repositories/services; a cross-module exception handler keeps uniform 4xx/5xx error envelopes.
- **Planned data relations:** the visitor journey can later link to machinery equipment shown on tours, plants grown during visits, and sitesecurity access records (a CredentialUser entity already exists in the sitesecurity module), feeding a coherent full-platform view.
- **Changing the platform without conflict:** because the shared frontend files stay untouched and the module keeps its endpoints local, merging VM into develop stays clean even while sibling branches are still open. Once the sibling modules merge, this branch's diff shrinks automatically to the visitor-management work only.

6. Authentication and roles (planned)

Current state: no authentication. The REST API is open - Spring Security, JWT, OAuth and any session handling are not present on the backend, and the frontend has no login flow, guard or access token. The module is fully functional but every endpoint and every screen is reachable by anyone who can reach the server. The

sidebar already displays a "logout" entry, anticipating the future session.

Planned model. When authentication is introduced (recommended approach: farm-wide SSO/identity provider issuing short-lived tokens, validated by a shared Spring Security layer), a logged-in user will be assigned one of three roles defined by the `StaffRole` enum: `GUIDE`, `RECEPTION` and `MANAGER`. Access is then restricted by role at two levels: backend (authorisation on every endpoint) and frontend (routing guards that hide or disable screens and actions).

6.1 Role-to-feature access matrix

Feature / screen	GUIDE	RECEPTION	MANAGER
Dashboard - KPIs & analytics			
Scheduling (slots)			
Registrations queue			
Visitors directory			
Educational program (stops/workshops)			
Safety briefing			
Bookings & activities			
Events			
Feedback / surveys			
Staff management			

6.2 Role descriptions

Role	Who	Typical access	Hidden / locked
GUIDE	Field staff leading visits	Dashboards for own tours, safety briefing delivery/signature, read-only view of registrations, slots, bookings, events, workshops	Staff management, payments, analytics, sensitive reports
RECEPTION	Front desk welcoming visitors	Registrations, check-in, visitors, slots, bookings, on-site feedback collection, event registrations, workshop management	Staff management, global analytics
MANAGER	Farm managers / admins	Everything: full dashboard & KPIs, publish events/workshops, payment handling, staff management, survey analytics, audit	Nothing content-wise; only platform/role administration

6.3 Implementation recommendations

- Backend: secure every `/api/v1` endpoint with method-level or path-based authorisation (e.g. `@PreAuthorize`) mapped to the three roles, plus a central `SecurityFilterChain` for token validation.

- Frontend: role-aware routing with guards; hide or disable menu entries and action buttons that the role cannot use, and re-check permissions on the API (never trust the UI alone).
- Keep the StaffRole enum as the single source of truth: GUIDE, RECEPTION, MANAGER, so the frontend can derive its role model from the same values the backend checks.
- Add role to env info/health or a small /api/v1/me endpoint so the frontend can load capabilities at startup.
- Long-term: link staff accounts to the farm-wide identity provider and let the sitesecurity CredentialUser entity carry the login credential for the whole platform.

Note: sections 6.1-6.3 describe the target state. They are provided as a blueprint to make role-based access a first-class part of the Visitor Management module once authentication lands in the platform.

7. How to run and verify

Prerequisites: Java 21, Node 20+, Docker (PostgreSQL 17). Load environment variables from .env (POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_PORT) before starting the backend.

Step	Command
Start PostgreSQL	<code>docker compose up -d postgres</code>
Start backend (no seed)	<code>./mvnw spring-boot:run</code>
Start backend (dev data)	<code>set -a; . ../.env; set +a; SPRING_PROFILES_ACTIVE=dev ./mvnw spring-boot:run</code>
Start frontend	<code>npm run dev</code> (open http://localhost:5173/visitormanagement)
Backend tests	<code>./mvnw test</code> (H2 profile, no PostgreSQL needed)
Frontend checks	<code>npm run lint && npm run build</code>
Health check	<code>curl http://localhost:8080/actuator/health</code>

Smoke examples: submit feedback with only a typed name (201), submit feedback without identity (422), delete an unattached visitor (204), delete a referenced visitor (422), create a slot with capacity 25 (201).

8. Delivery status

Item	Status
Backend REST API (visitors, registrations, slots, bookings, events, workshops, tour stops, feedback, staff, dashboard)	Delivered & tested
Frontend module shell, sidebar logo circle, 10 screens	Delivered
Dashboard with weekly KPIs and animated counters	Delivered
Entrance animations + reduced-motion support	Delivered
Merge with develop (shared endpoints, index.css palette)	Aligned, no conflicts
Authentication (roles GUIDE / RECEPTION / MANAGER)	Planned - blueprint in section 6